The typical sharding topology includes, at least, two separate data centers.

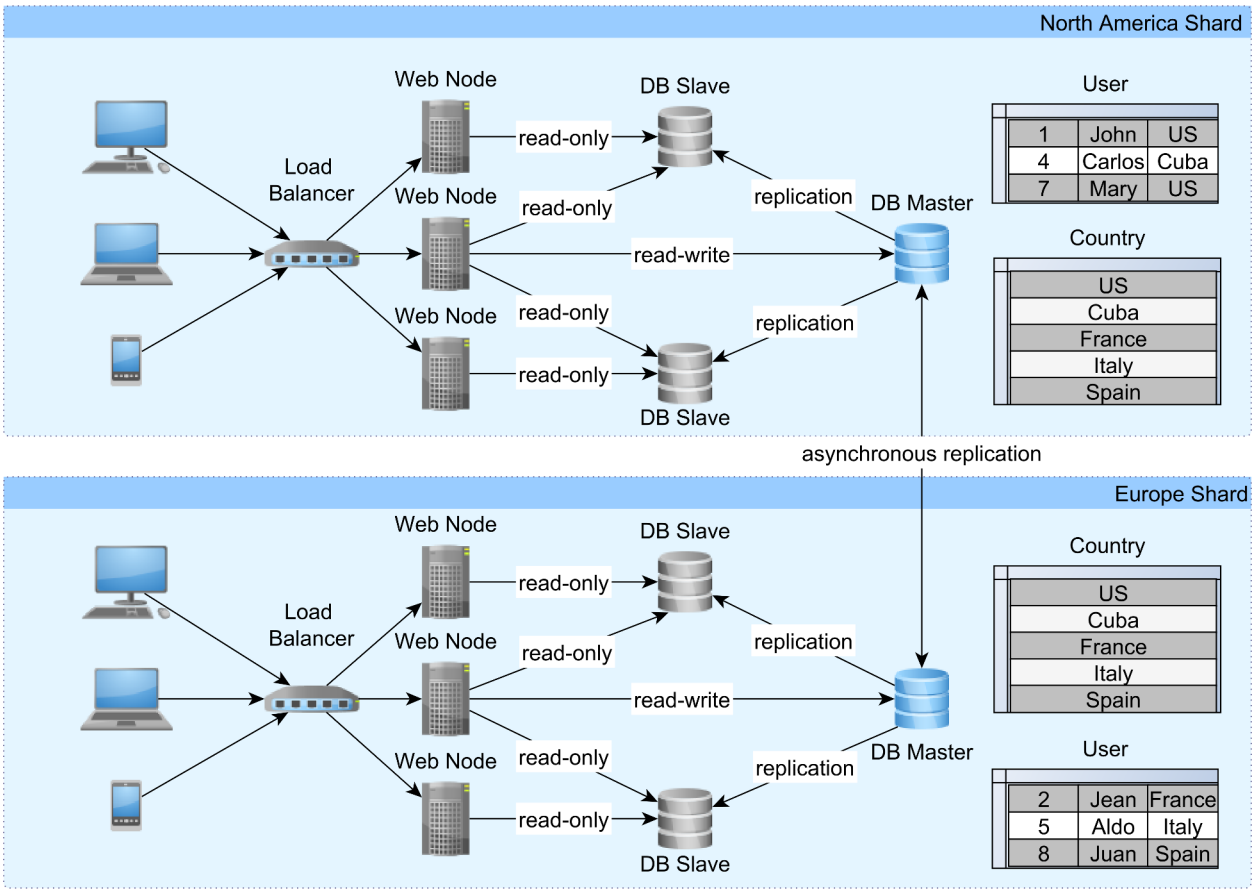

Figure 1.4: Sharding

Each data center can serve a dedicated geographical region, so the load is balanced across geographical areas. Not all tables need to be partitioned across shards, smaller size ones being duplicated on each partition. To keep the shards in sync, an asynchronous replication mechanism can be employed.

In the previous diagram, the country table is mirrored from one data center to the other, and partitioning happens on the user table only. To eliminate the need for inter-shard data processing, each user along with all user-related data are contained in one data center only.

In the quest for increasing system capacity, sharding is usually a last resort strategy, employed after exhausting all other available options, such as:

- optimizing the data layer to deliver lower transaction response times
- scaling each replicated node to a cost-effective configuration
- adding more replicated nodes until synchronization latencies start dropping below an acceptable threshold.

MySQL cluster auto-sharding

MySQL Cluster[a] offers automatic sharding, so data is evenly distributed (using a primary key hashing function) over multiple commodity hardware machines. Every node accepts both read and write transactions and, just like Multi-Master replication, conflicts are automatically discovered and resolved.

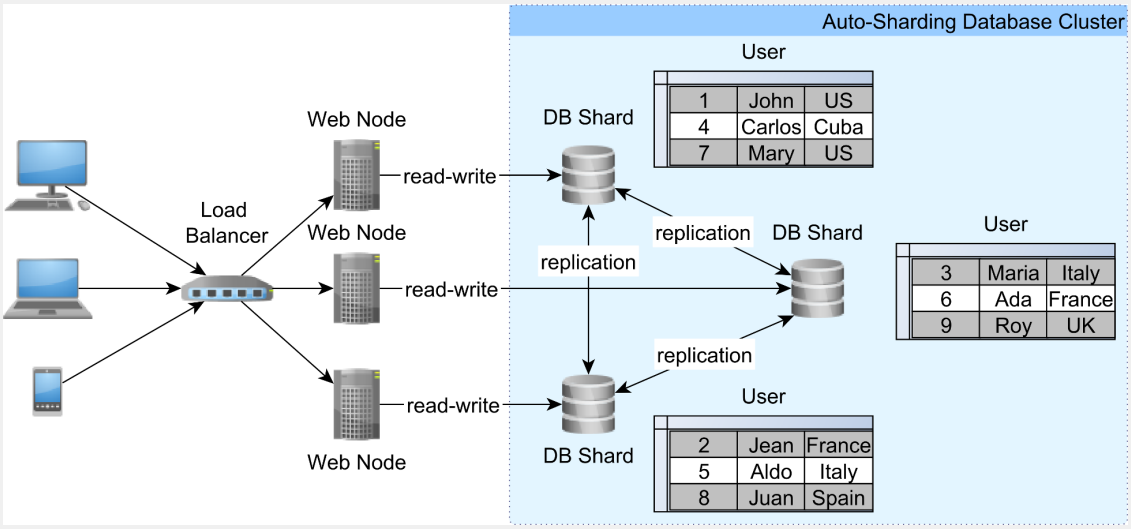

Figure 1.5: Auto-sharding

The auto-sharding topology is similar to the Multi-Master replication architecture as it can increase throughput by distributing incoming load to multiple machines. While in a Multi-Master replicated environment every node stores the whole database, the auto-sharding cluster distributes data so that each shard is only a subset of the whole database.

Because the cluster takes care of distributing data, the application does not have to provide a data shard routing layer, and SQL joins are possible even across different shards. MySQL Cluster 7.3 uses the *NDB* storage engine, and so it lacks some features provided by InnoDB[b] like multiple transaction isolation levels or MVCC (Multi-Version Concurrency Control).

[a] https://www.mysql.com/products/cluster/scalability.html
[b] http://dev.mysql.com/doc/mysql-cluster-excerpt/5.6/en/mysql-cluster-ndb-innodb-engines.html

Little's Law

In any given system, the ultimate relationship between response time and throughput is given by Little's Law[a] and, high values of incoming throughput can cause an exponential growth in response time due to resource saturation.

Nevertheless, when taking a single database connection, by lowering the average transaction response time, more transactions can be accommodated in a given time unit. For this reason, the following chapters explain in greater detail what is needed to be done in order to reduce transaction response time as much as possible.

[a] https://people.cs.umass.edu/~emery/classes/cmpsci691st/readings/OS/Littles-Law-50-Years-Later.pdf

2. JDBC Connection Management

The JDBC (Java Database Connectivity) API provides a common interface for communicating with a database server. All the networking logic and the database-specific communication protocol are hidden away behind the vendor-independent JDBC API. For this reason, all the JDBC interfaces must be implemented according to the database vendor-specific requirements. The `java.sql.Driver` is the main entry point for interacting with the JDBC API, defining the implementation version details and providing access to a database connection.

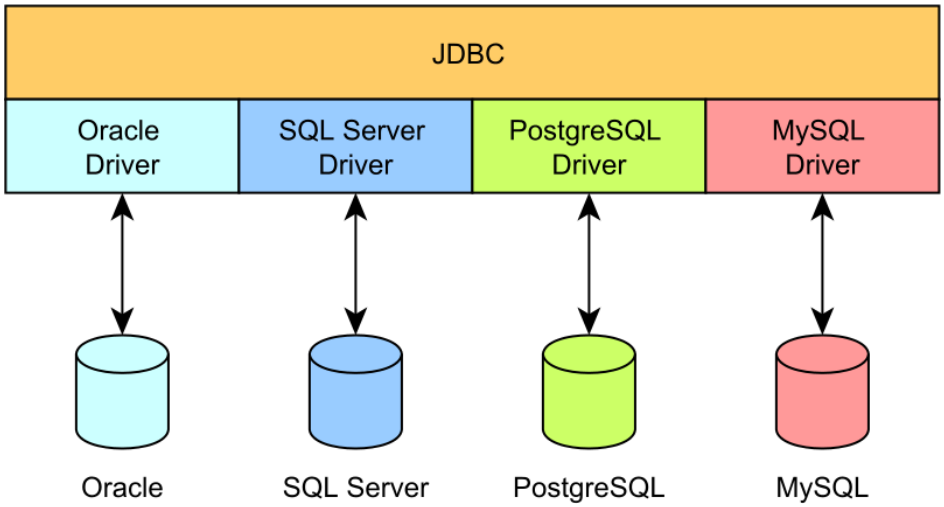

Figure 2.1: JDBC plugin architecture

JDBC defines four driver types:

- Type 1: It is only a bridge to an actual ODBC driver implementation.
- Type 2: It uses a database-specific native client implementation (e.g. *Oracle Call Interface*).
- Type 3: It delegates calls to an application server offering database connectivity support.
- Type 4: The JDBC driver implements the database communication protocol solely in Java.

Being easier to setup and debug, the Type 4 driver is usually the preferred alternative.

To communicate with a database server, a Java program must first obtain a `java.sql.Connection`. Although the `java.sql.Driver` is the actual database connection provider, it is more convenient to use the `java.sql.DriverManager` since it can also resolve the JDBC driver associated with the current database connection URL.

> Previously, the application required to load the driver prior to establishing a connection but, since JDBC 4.0, the *Service Provider Interfaces* mechanism can automatically discover all the available drivers in the current application classpath.

2.1 DriverManager

The `DriverManager` defines the following methods:

```
public static Connection getConnection(
        String url, Properties info) throws SQLException;

public static Connection getConnection(
        String url, String user, String password) throws SQLException;

public static Connection getConnection(
        String url) throws SQLException;
```

Every time the `getConnection()` method is called, the `DriverManager` requests a new *physical* connection from the underlying `Driver`.

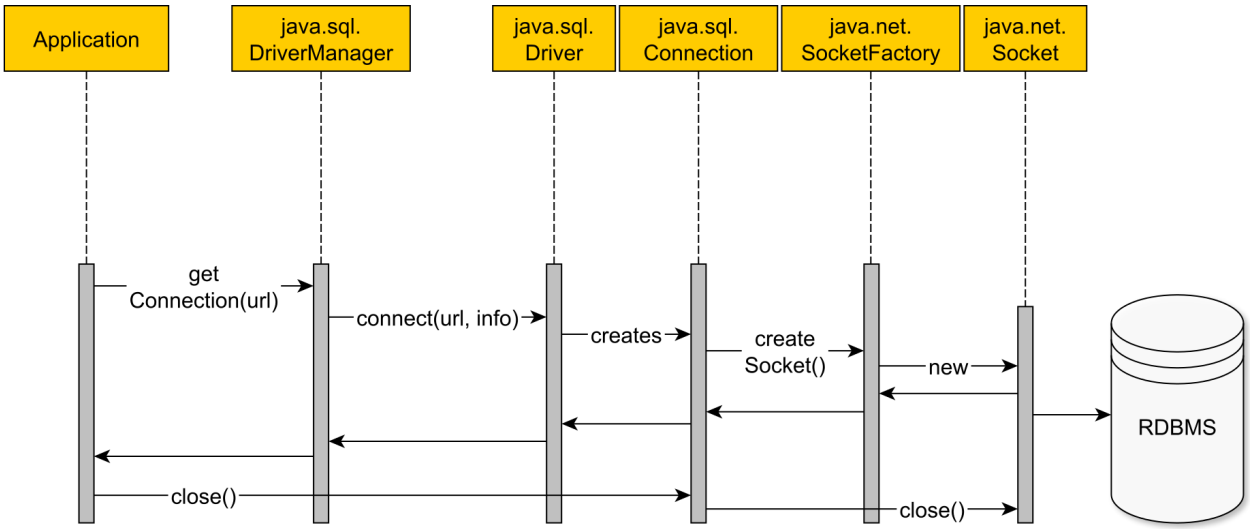

Figure 2.2: DriverManager connection

The first version of JDBC was launched in 1997, and it only supported the `DriverManager` utility for fetching database connections. Back then, Java was offering support for desktop applications which were often employing a two-tier architecture: